The LSTM-PER-TD3 Algorithm for Deep Reinforcement Learning in Continuous Control Tasks

1st Qurui Zhang
School of Computer Science
Nanjing University of Posts and Telecommunications
Nanjing 210023, China
ruimatthew@163.com

2nd Lirui Zhang
School of Internet of Things
Nanjing University of Posts and Telecommunications
Nanjing 210023, China
roylee4612@163.com

3rd Qi Ma
School of Computer Science
Nanjing University of Posts and Telecommunications
Nanjing 210023, China
maqi0618@163.com

4th Jing Xue
School of Computer Science
Nanjing University of Posts and Telecommunications
Nanjing 210023, China
xuejing@njupt.edu.cn

2023 China Automation Congress (CAC) | 979-8-3503-0375-9/23/$31.00 ©2023 IEEE | DOI: 10.1109/CAC59555.2023.10450268

***Abstract*—Deep reinforcement learning has shown great potential in the field of robot control, but it still faces challenges in continuous control tasks. Traditional reinforcement learning algorithms perform poorly when dealing with high-dimensional state spaces and continuous action spaces, resulting in low learning efficiency and degraded performance. To solve this issue, we propose a novel reinforcement learning algorithm named LSTM-PER-TD3 (LPT3), which combines Long Short-Term Memory networks (LSTM), prioritized experience replay (PER), and Twin Delayed Deep Deterministic Policy Gradient algorithm (TD3). The LPT3 algorithm utilizes LSTM networks to process sequential state information and combines PER and TD3 methods to achieve efficient continuous control. It is capable of learning accurate policies in high-dimensional state spaces, improving learning efficiency and performance through experience replay and policy evaluation. We evaluate the LPT3 algorithm on several standard continuous control tasks and compare it with traditional reinforcement learning algorithms. Experimental results demonstrate the superiority of LPT3 in terms of learning efficiency and performance. Compared to traditional algorithms, LPT3 converges faster and achieves higher control accuracy, effectively addressing the challenges in continuous control tasks.**

***Index Terms*—LSTM, TD3, PER, Deep reinforcement learning**

I. Introduction

Deep reinforcement learning, as a powerful machine learning method, has been widely applied in robot control. In robot control, continuous control [1, 3] is an important task that requires the robot to continuously observe and learn from the environment to obtain better control strategies. However, traditional reinforcement learning algorithms cannot be directly applied to continuous control tasks because the actions in continuous action spaces are real-valued rather than discrete values.

Researchers have proposed many deep learning-based continuous control algorithms in recent years. Algorithms based on actor-critic networks can more effectively handle high-dimensional state and action spaces. S. Fujimoto [4] proposed the TD3 algorithm which is a very popular algorithm that improves learning efficiency and stability by using double Q-networks and a delayed update strategy. However, the TD3 algorithm still has some problems, such as convergence issues when dealing with high-dimensional state and action spaces [5], and low sample efficiency.

As an important sequence model, LSTM [6] can help deep reinforcement learning algorithms better handle continuous control tasks. Meng L [7] proposed the LSTM-TD3 algorithm, which is a variant of TD3 algorithm that uses LSTM to capture historical states and actions. The main advantage of LSTM-TD3 is that it can handle partially observable environments. This enables LSTM-TD3 to handle reward signals with long time delays better. However, LSTM-TD3 also has some drawbacks. Due to the complexity of LSTM, it requires more computational resources and time to train. To further improve the performance of continuous control algorithms, researchers have proposed many improved algorithms. For example, algorithms based on prioritized experience replay [8] can improve sample efficiency.

Although some progress has been made, there are still many challenges in applying deep reinforcement learning to robot control. On the one hand, robot control tasks typically involve high-dimensional state and action spaces that require a large amount of computational resources and time. On the other hand, the construction and control of actual robot systems must also take into account the hardware characteristics of the robot and the complexity of the real-world environment.

Authorized licensed use limited to: University of Mary Washington. Downloaded on July 15,2025 at 15:08:10 UTC from IEEE Xplore. Restrictions apply.

Therefore, designing efficient, stable, and robust deep reinforcement learning algorithms, and applying them to actual robot systems, remains a challenging problem.

Inspired by these observations, this paper proposes a memory-based deep reinforcement learning [9]. The algorithm combines prioritized experience replay with LSTM networks to improve learning efficiency and stability in continuous robot control tasks. We compare our proposed algorithm with mainstream reinforcement learning algorithms, and the results show that our algorithm outperforms the baseline methods in terms of average return, demonstrating its superior performance. Therefore, incorporating LSTM-based memory and prioritized experience replay into the TD3 algorithm can enhance the agent's ability to handle long-term dependencies and experience transitions.

II. Background

A. Deep Reinforcement Learning (DRL)

DRL [9] is a learning paradigm that builds upon the foundations of reinforcement learning and utilizes deep neural networks to learn the end-to-end value function and policy. Reinforcement learning enables an agent to learn how to act in an environment without prior knowledge of the transition and reward models by trial and error. At a discrete time t, the agent interacts with the external environment by choosing either a stochastic policy $\pi(a_t|s_t)$ or a deterministic policy $a_t = \mu(s_t)$ based on the current state s_t Through continuous interaction with the environment and receiving new states and rewards, the agent learns an optimal policy to maximize the expected future return. Along with the policy π or μ, a state-value function $V(s)$ and action-value function $Q(s, a)$ are commonly learned. For MDP, the state-value function $V_{(s)}$ represents the expected return from state s under policy π, while the action-value function $Q_\pi(s, a)$ represents the expected return from state s, taking action a, and following policy π thereafter. Traditional reinforcement learning methods use tabular representation or simple function approximation with hand-crafted feature construction to represent the value function, whereas DRL learns the value function and policy in an end-to-end manner without feature engineering by utilizing deep neural networks.

B. Long Short Term Memory (LSTM)

LSTM [6] is a type of recurrent neural network architecture designed to solve the vanishing and exploding gradient problems in conventional RNNs. It consists of a special memory cell that contains an input gate, a forget gate, and an output gate to control the flow of information in and out. The memory cell also includes a cell state to capture long-term dependencies in sequences. Computation in LSTM is controlled by gate mechanisms that can adaptively learn which information should be retained or discarded. These gates use sigmoid activation functions to decide which information should be forgotten or kept. This gate mechanism enables LSTM to handle long sequences and long-term dependencies, resulting in better performance compared to traditional RNNs.

C. Prioritized Experience Replay (PER)

PER [8] is an algorithm used in reinforcement learning that improves the efficiency and stability of learning. Based on experience replay, PER assigns different sampling probabilities to each sample so that more critical samples are sampled more frequently, thus improving the efficiency and stability of the algorithm.

D. Twin Delayed Deep Deterministic Policy Gradient (TD3)

The TD3 algorithm [4] is a reinforcement learning algorithm designed for continuous action spaces and is an improved version of the DDPG algorithm [10]. Building upon the DDPG algorithm, the TD3 algorithm adopts the Double Q Learning approach and employs actor-critic mechanism to calculate target values for two Q functions. Compared to the DDPG algorithm, the TD3 algorithm has the following improvements:

- Two critic networks are employed, each with a target network, resulting in a total of four networks.
- The delayed policy updates strategy is employed, which updates the actor network every certain period of time.
- Target policy smoothing is employed, which adds noise to the actor's output when computing the target.

III. The LSTM-PER-TD3 Algorithm

The LSTM-PER-TD3 Algorithm (LPT3) is a deep reinforcement learning architecture that combines LSTM networks with PER and the TD3 algorithm. In TD3, experiences in the experience pool are randomly selected, which may result in important and rare experiences being overlooked because the probability of selecting a particular experience is proportional to its frequency in the pool, rather than its importance to the learning process. As a result, informative experiences that could have a significant impact on the agent's performance may be underutilized. This issue is particularly pronounced in continuous control tasks, where good experiences are rare and the exploration process is slow. Prioritized experience replay addresses this limitation by assigning a priority to each experience based on its expected learning value and sampling transitions with higher priorities more frequently. By focusing on informative experiences and giving them more weight in the training process, PER can significantly improve the efficiency and stability of the learning process. However, the probability of replaying a transition increases with its Temporal Difference errors (TD-error), which can lead to overfitting and instability. Therefore, we incorporate recurrent neural network (RNN) structures [11], in this paper we use LSTM networks, in the actor and two critic networks to better capture the dependencies between state sequences and model sequential data. This is particularly important for continuous action control tasks such as robotic motion control and speech recognition. The memory units in LSTM networks can retain and process information from previous states, leading to more accurate decision-making in the current state. Additionally, using RNNs can improve the model's generalization performance and alleviate the impact of discontinuities in the data

Authorized licensed use limited to: University of Mary Washington. Downloaded on July 15,2025 at 15:08:10 UTC from IEEE Xplore. Restrictions apply.

distribution. The overall structure of the algorithm is shown in Fig. 1.

A. Experience pool using PER

Experience pool with Prioritized Experience Replay is a variant of the traditional experience replay algorithm used in reinforcement learning. In traditional experience replay, the agent stores transitions in a buffer and randomly samples transitions from the buffer during training. However, this method does not consider the importance of each transition in improving the agent's performance.

PER solves this problem by assigning a priority value to each transition in the buffer, which reflects its importance in learning. The priority of a transition is determined based on the magnitude of its TD-error, which is a measure of the difference between the agent's predicted Q-value and the target Q-value. Transitions with higher TD-error are assigned higher priorities and are more likely to be sampled during training.

The update equation for PER can be given by (1):

$$w_i = \frac{(1/N \cdot P(i))^\beta}{\max_j (1/N \cdot P(j))^\beta} \tag{1}$$

where w_i is the sampling weight of the ith transition, $P(i)$ is the priority of the ith transition, which defined as (2), N is the size of the buffer, and β is a hyperparameter that controls the degree of prioritization. The higher the value of β, the more algorithm prioritizes transitions with high priority.

$$p_i = \frac{1}{1 + \epsilon \cdot \frac{P(i)}{\max_j P(j)}} \tag{2}$$

Here, ϵ is a hyperparameter that controls the degree of exploration and ensures that all transitions have a non-zero probability of being selected.

During the iterative process, when the number of experiences in the experience pool exceeds N, the agent samples transition from the buffer using the above update equation to update its Q-network. After the training is complete, the priorities of the transitions in the buffer are updated based on their TD-errors, and the process repeats. This method has been shown to improve the sample efficiency and learning speed of the agent. The TD-error could be computed by (3):

$$\delta_t = r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \tag{3}$$

Where δ_t is the TD-error at time step t, r_t is the reward received at time step t, γ is the discount factor, $Q(s_t, a_t)$ is the estimated Q-value of the current state-action pair, and $\max_{a'} Q(s_{t+1}, a')$ is the estimated maximum Q-value of the next state s_{t+1} over all possible actions a'.

B. LSTM Actor-Critic Framework

In LSTM, the input sequence is fed as a series of vectors, each representing an element of the sequence. The history vector is responsible for capturing the relevant past information from the input sequence. In the case of a history vector with length l, it would represent the l most recent elements of the input sequence. The h_t^l can be expressed as (4).

$$h_t^l = (s_{t-l}, a_{t-l}, \cdots, s_{t-1}, a_{t-1}) \tag{4}$$

Where s_{t-l} and a_{t-l} represent the action and state taken at time $(t-l)$, s_{t-1} and a_{t-1} represent the action and state taken at time $(t-1)$.

In the current work, we adopt the memory-based critic Q to evaluate the Q-value of a given state-action pair. The critic Q and actor μ can be seen as a compound function of the memory extraction Q^{me} and μ^{me}, the current feature extraction Q^{cf} and μ^{cf}, and the perception integration Q^{pi} and μ^{pi} components, following (5) and (6):

$$\begin{aligned} Q\left(s_t, a_t, h_t^l\right) &= Q^{me} \circ Q^{cf} \circ Q^{pi} \\ &= Q^{pi}\left(Q^{me}\left(h_t^l\right) \bowtie Q^{cf}\left(s_t, a_t\right)\right) \end{aligned} \tag{5}$$

$$\begin{aligned} \mu\left(s_t, h_t^l\right) &= \mu^{me} \circ \mu^{cf} \circ \mu^{pi} \\ &= \mu^{pi}\left(\mu^{me}\left(h_t^l\right) \bowtie \mu^{cf}\left(s_t\right)\right) \end{aligned} \tag{6}$$

Here, $\bowtie$ indicates the concatenation operation, $\circ$ denotes function composition, Q^{me} and μ^{me} are the extracted memory based on the history vector h_t^l, Q^{cf} and μ^{cf} are the extracted current feature based on the current state s_t and action a_t. These two equations are adopted from [7].

C. The design of the optimization

For each critical (Q_1, Q_2), the optimization aims to minimize the mean squared error between the predicted Q and the estimated target $\hat{Q}$ with respect to the parameters θ^Q of the critical Q, which can be expressed as (7).

$$\min_{\theta^Q} \mathbb{E}_{\left\{\left(h_t^l, s_t, a_t, r_t, s_{t+1}, d_t, p_t\right)_i\right\}_{i=1}^N} \left(p * \left(Q - \hat{Q}\right)\right)^2 \tag{7}$$

Where $\hat{Q}$ can be caculated by (8).

$$\hat{Q} = r_t + \gamma * (1 - d_t) * \min_{j=1,2} Q_j^-\left(s_{t+1}, a^-, h_{t+1}^l\right) \tag{8}$$

The actor network aims to maximize the expected return based on the state and latent variable h_t^l by optimizing the parameters θ^μ of the actor network. The optimization is performed by taking the action that maximizes the predicted Q value from the critic networks. The expression for the optimization is shown as (9):

$$\max_{\theta^\mu} \mathbb{E}_{\left\{\left(h_t^l, s_t\right)_i\right\}_{i=1}^N} Q\left(s_t, \mu\left(s_t, h_t^l\right), h_t^l\right) \tag{9}$$

where $Q(s_t, \mu(s_t, h_t^l), h_t^l)$ is the predicted Q value for the current state s_t and latent variable h_t^l given the action selected by the actor network $\mu(s_t, h_t^l)$. The optimization aims to find the optimal policy that maximizes the expected return over a set of training data.

Authorized licensed use limited to: University of Mary Washington. Downloaded on July 15,2025 at 15:08:10 UTC from IEEE Xplore. Restrictions apply.

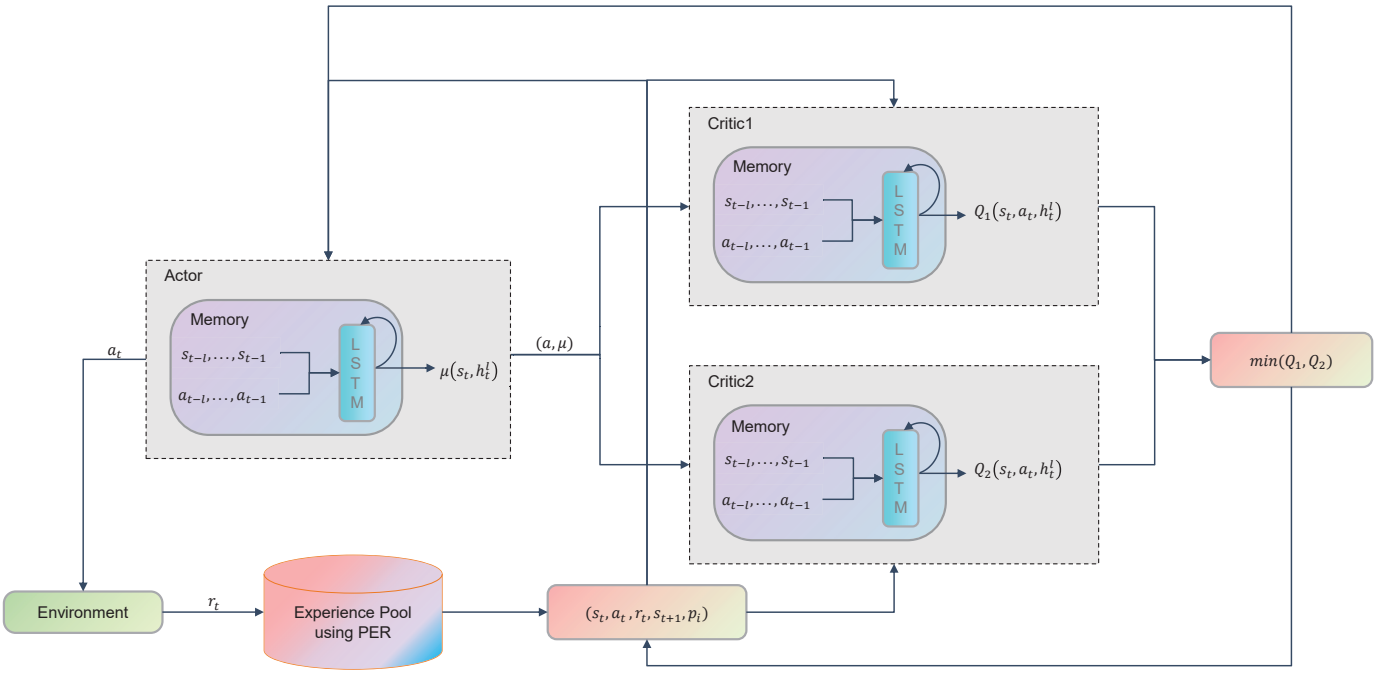
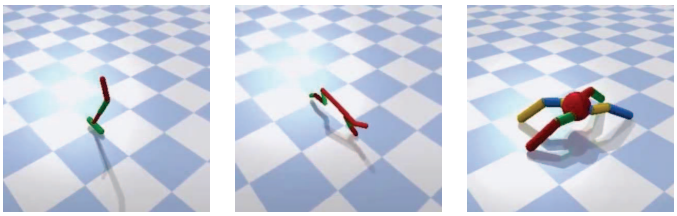


Fig. 1. The structure of LPT3 algorithm.

IV. Experiment

We conducted extensive experiments to evaluate the performance of our proposed LSTM-PER-TD3 (LPT3) algorithm on three benchmark tasks, namely AntBulletEnv-v0, HalfCheetahBulletEnv-v0, and HopperBulletEnv-v0, as shown in Fig. 2. Compared with several state-of-the-art algorithms, including Soft Actor-Critic (SAC) [12], Deep Deterministic Policy Gradient (DDPG) [10], Twin Delayed DDPG (TD3) [4], and LSTM-TD3 [7].

- In HopperBulletEnv-v0, the agent controls a one-legged hopper with the goal of jumping as far as possible.
- In HalfCheetahBulletEnv-v0, the agent controls a two-legged cheetah with the goal of running as fast as possible.
- In AntBulletEnv-v0, the agent controls a four-legged ant with the goal of moving forward as fast as possible.



a.HopperBulletEnv-v0 b.HalfCheetahBulletEnv-v0 c.AntBulletEnv-v0

Fig. 2. BulletGym tasks

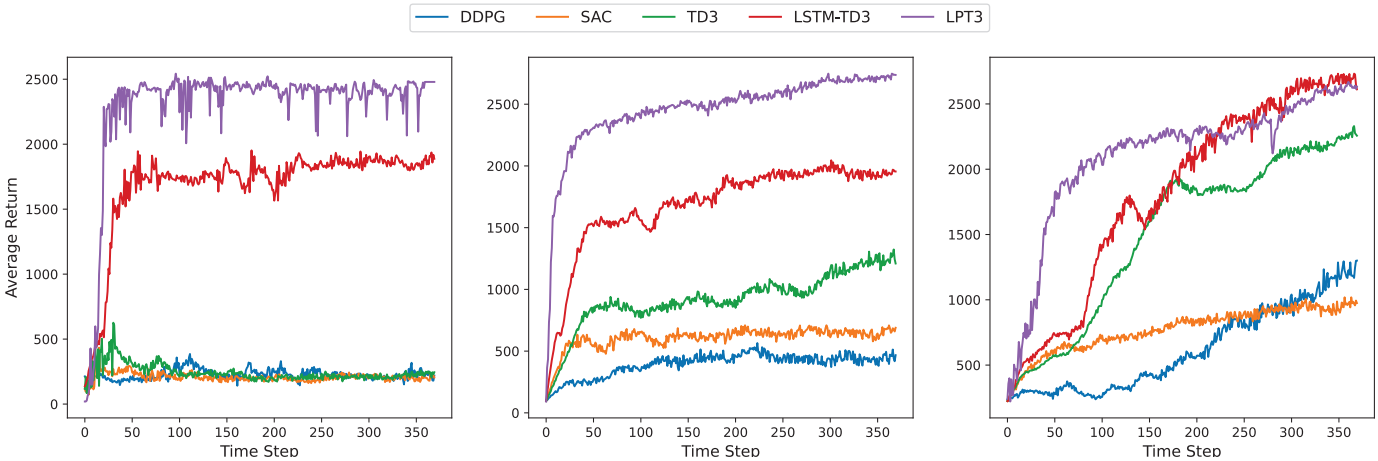
We used the Bullet3 physics engine[1] implemented in the OpenAI GYM MuJoCo environment based on bulletphysics.

[1]https://github.com/bulletphysics/bullet3

In each task, we randomly initialized the agents and conducted experiments for 5 runs with different seeds. For each run, we trained the agents and the results were averaged over the 5 runs. The results are presented in Fig. 3.

At the time of experimentation, we observed a distinct pattern in the algorithm's performance through a systematic analysis by varying the history length from 1 to 10. The specific results are shown in TABLE I. In the HopperBulletEnv-v0 and AntBulletEnv-v0 tasks, the algorithm demonstrated a consistent improvement as the history length increased up to $l = 5$, where it achieved the highest average return of 2542.92 and 2670.30, respectively. This finding suggests that incorporating a moderate amount of historical information effectively captures relevant temporal dependencies, leading to enhanced decision-making in these tasks. However, in the HopperBulletEnv-v0 environment, although there was a slight improvement when increasing the history length to 10 (2373.51 compared to 2542.92 at $l = 5$), the difference was not substantial. This observation indicates that while a longer history length may provide some benefit in terms of performance, the gain is limited in this particular task. Moreover, it is worth noting that as the history length exceeds the optimal point of $l = 5$, the algorithm's performance tends to plateau or even decline across all three benchmark tasks. This suggests that excessively long histories introduce noise and irrelevant information, hampering the algorithm's ability to extract meaningful insights for decision-making. In conclusion, the experimental findings conclusively demonstrate that a history length of 5 is the most suitable choice for our LPT3 algorithm. This length allows the algorithm to effectively capture relevant temporal dependencies while avoiding

Authorized licensed use limited to: University of Mary Washington. Downloaded on July 15,2025 at 15:08:10 UTC from IEEE Xplore. Restrictions apply.



a.HopperBulletEnv-v0 b.HalfCheetahBulletEnv-v0 c.AntBulletEnv-v0

Fig. 3. Comparison of Average Return of LPT3 and other DRL methods in different tasks.

TABLE I
PERFORMANCE OF LPT3 WITH DIFFERENT HISTORY LENGTHS.

	HopperBulletEnv-v0				**HalfCheetahBulletEnv-v0**				**AntBulletEnv-v0**			
	$l=1$	$l=3$	$l=5$	$l=10$	$l=1$	$l=3$	$l=5$	$l=10$	$l=1$	$l=3$	$l=5$	$l=10$
Average Return	227.84	2433.60	**2542.92**	2373.51	899.77	2524.31	2776.56	**2783.17**	874.68	1798.23	**2670.30**	2591.45

TABLE II
RESULTS OF THE EXPERIMENTS ON THE THREE TASKS.

	DDPG	**SAC**	**TD3**	**LSTM-TD3**	**LPT3**
HopperBulletEnv-v0	355.28	362.58	596.21	1931.11	**2542.92**
HalfCheetahBulletEnv-v0	261.21	745.08	1380.29	2032.44	**2776.56**
AntBulletEnv-v0	1278.49	1072.47	2479.22	**2739.85**	2670.30

the negative effects of noise and irrelevant information. By selecting the optimal history length, our algorithm achieves superior performance in challenging continuous control tasks, showcasing its robustness and effectiveness.

In this paper, We set the discount factor γ to 0.99. The policy update delay was set to 2, and the target actor noise was set to 0.2. The batch size for both actor and critic was set to 256, and the replay buffer size was set to 10^6, we used the Adam optimizer with a learning rate of 10^{-3} for both actor and critic networks.

The results are presented in TABLE II, where the maximum average return of the time step return over the 5 runs is reported. Our proposed LPT3 algorithm achieved the highest mean time step return in two tasks, namely HalfCheetahBulletEnv-v0 and HopperBulletEnv-v0.

The superior performance of LPT3 can be attributed to several key factors. Firstly, the algorithm leverages a recurrent neural network to model the dynamics of the environment, allowing it to capture long-term dependencies and effectively utilize historical information. This is particularly beneficial in continuous control tasks where past states can provide valuable context for decision-making.

Secondly, LPT3 incorporates prioritized experience replay, which prioritizes the replay of experiences based on their importance in the learning process. By focusing on experiences with higher temporal difference errors, LPT3 accelerates learning and improves sample efficiency. This prioritization mechanism enables the algorithm to make more effective use of the available data and enhances its overall performance.

Lastly, LPT3 employs a target network for stability during training. The target network helps mitigate the issue of moving target distributions by providing a consistent target for the value function estimation. This stabilizes the learning process and improves the algorithm's convergence properties.

Overall, the experimental results demonstrate the effectiveness of our proposed LPT3 algorithm, which can achieve superior performance compared with several state-of-the-art reinforcement learning algorithms on challenging continuous control tasks. These results not only underscore the superiority of LPT3 in continuous control tasks but also emphasize the importance of incorporating historical information and utilizing prioritized experience replay in reinforcement learning algorithms. The findings indicate that leveraging a moderate amount of past experiences, combined with prioritized replay, can lead to significant performance improvements. However, careful consideration is required when selecting the

Authorized licensed use limited to: University of Mary Washington. Downloaded on July 15,2025 at 15:08:10 UTC from IEEE Xplore. Restrictions apply.

history length, as excessively long histories may introduce noise and irrelevant information, hampering the algorithm's performance. The experiments confirm the effectiveness of incorporating historical information and prioritized experience replay, highlighting the importance of selecting an appropriate history length for achieving optimal performance in continuous control scenarios.

In TABLE II, our proposed method did not achieve the best performance in the AntBulletEnv-v0 environment. This observation can be attributed to the inherent complexity and challenges associated with the AntBulletEnv-v0 task. The state space of the AntBulletEnv-v0 task may possess intricate structures and a high degree of variability, comprising numerous state variables. Handling such a high-dimensional and diverse state space necessitates strategies that are more precise and efficient to explore and find optimal solutions. Conventional reinforcement learning algorithms might face limitations in dealing with this complexity, leading to performance degradation.

V. Conclusion

In this paper, we proposed a new algorithm called LPT3 for continuous control tasks, which combines LSTM and TD3 with a novel policy learning mechanism. We conducted experiments on three benchmark environments using various state-of-the-art reinforcement learning algorithms, including DDPG, SAC, TD3, LSTM-TD3, and our proposed LPT3 algorithm. The experimental results show that our proposed LPT3 algorithm outperforms the other baseline algorithms on two out of three tasks, namely HalfCheetahBulletEnv-v0 and HopperBulletEnv-v0. However, it is slightly inferior to LSTM-TD3 on the AntBulletEnv-v0 task.

Our experiments also demonstrate the effectiveness of the proposed policy learning mechanism, which shows that the learned policy improves over time and converges faster than the original policy. In addition, the experiments also show that using LSTM as a memory module can significantly improve the performance of TD3. The LPT3 algorithm can be used as a baseline for further research in continuous control tasks, including but not limited to robotics control. The ability of LPT3 to learn from past experience and improve the policy over time can significantly enhance the performance of robots in real-world applications. Additionally, the combination of LSTM and TD3 can be used as a memory module to capture the temporal dynamics of robot movements and achieve better control accuracy. Therefore, LPT3 has the potential to be a valuable tool for researchers and engineers working in the field of robotics and control systems. Future work can explore the applicability of LPT3 in other domains and further optimize the algorithm for improved performance.

Acknowledgment

Supported in part by the Major Project of Philosophy and Social Science Research in Jiangsu Universities of China (2020SJZDA102).

References

[1] J. Zhang and Y. Chen, "Application of deep reinforcement learning in robot control," Robot Technology and Application, vol. 5, pp. 1-7, 2019.

[2] X. F. Li and P. F. Wang, "Application of deep reinforcement learning in robot control," Electronic Design Engineering, vol. 10, pp. 1-4, 2020.

[3] X. F. Ma, "Application of deep reinforcement learning in robot control," Science and Technology Innovation and Application, vol. 12, pp. 1-2, 2020.

[4] S. Fujimoto, H. Van Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," arXiv preprint arXiv:1802.09477, 2018.

[5] Liu, J., Wang, Y. On the Convergence of Reinforcement Learning in Nonlinear Continuous State Space Problems. IEEE Transactions on Neural Networks and Learning Systems, 31(2), 1-13.

[6] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[7] Meng L, Gorbet R, Kulić D. Memory-based deep reinforcement learning for pomdps[C]//2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE, 2021: 5619-5626.

[8] Schaul Tom, Quan John, Antonoglou Ioannis, and Silver David. Prioritized experience replay. arXiv preprint arXiv:1511.05952, 2015.

[9] V . Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, "Playing atari with deep reinforcement learning," arXiv preprint arXiv:1312.5602, 2013.

[10] T. P . Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y . Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," arXiv preprint arXiv:1509.02971, 2015.

[11] Zaremba W, Sutskever I, Vinyals O. Recurrent neural network regularization[J]. arXiv preprint arXiv:1409.2329, 2014.

[12] T. Haarnoja, A. Zhou, P . Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in International Conference on Machine Learning. PMLR, 2018, pp. 1861–1870.

Authorized licensed use limited to: University of Mary Washington. Downloaded on July 15,2025 at 15:08:10 UTC from IEEE Xplore. Restrictions apply.